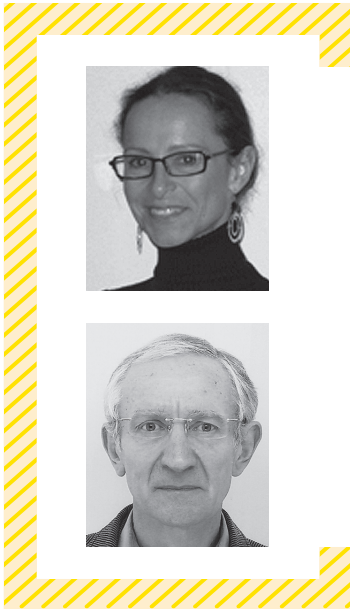




Requirements Tracery

Olly Gotel and Stephen Morris



WHAT DO YOU see when you look at Figure 1? Our guess is that you see a spectacular rose window, one common to many medieval cathedrals. You know that the whole stained glass window is comprised of many pieces of colored glass, each grouped within much smaller constructs often called leaded lights. Daylight passes through each of these lights to form its own small pattern, sometimes figurative, sometimes abstract. What is seldom immediately appreciated is the tracery of stone that holds each of these sources of light—sub-elements of the whole window design—in place and in a particular position. It's this supporting tracery that facilitates the complex overall design and imagery. The tracery is an integral part of the complete window and is perforce the first part of the window that architects design and put in place.

Requirements resemble these individual glass lights. While they might have some sig-

nificance in isolation, it's through their interrelated combinations of meaning that we gain a comprehensive picture of what a software system should do. Not only do we pay little attention to crafting any essential supporting structure in software engineering, we don't even bother to name it.

Tracery and Traceability

Tracery is a standard architectural term describing the stonework structure that supports the glass lights in a Gothic window.¹ Used metaphorically, the term is associated with something delicate, difficult to construct, and easily broken. Requirements fit this description quite nicely, as they can be challenging to determine, open to multiple interpretations, and subject to changes once specified. Yet, requirements are often woven together without thought to any supporting tracery. This omission is particularly relevant in the context of software engineering because these requirements should inform the development of all the consequent artifacts that will eventually realize the requirements in a working system. Therefore, to provide for ongoing support as requirements become implemented and evolve, the associated tracery must interweave myriad additional artifact types.

Traceability, a term common in software engineering, isn't tracery. Traceability expresses the potential to create and follow



FIGURE 1. Looking up in the south transept of the Cathédrale Notre-Dame de Laon, France. The tracery of stone holds together individual leaded lights of colored glass to form the overall design of a Gothic rose window. (Photo courtesy of John Ferro Sims.)

links between pairs of development artifacts to support analytical or development tasks. While a traceability information model or some similar specification of permissible links between artifact types can predefine the intended types of trace to establish and use, the overall tracery obtained in practice is rarely designed or scrutinized. The sum total of the actual traces created within a project provides the scaffolding for supporting many diverse engineering activities. They deserve far more attention.

Beyond the Matrix

A widely used method to examine traces in practice is to construct a traceability matrix that shows how artifacts of one type link to artifacts of another type, with the semantics of this link often qualified in some way. For instance, the matrix in Figure 2a reveals which requirements depend upon which others, while the matrix in Figure 2b reveals which requirements are satisfied by which elements of the design. While they play a valuable role in software engineering, such matrices only provide for very overt and somewhat partial views of tracery.

The view of the tracery in Figure 2a is overt because it's problematic to recognize the overall requirements structure; it's like looking at a rose window and seeing little more than stone ribs. The tracery of Figure 2b is partial because such matrices must be placed in series to examine the full trace of any one requirement through all life-cycle phases of a software system's development. Moreover, as the number of artifacts and links increases, individual matrices become unwieldy to create, maintain, and use, let alone to aggregate.

The traceability community has long recognized the need for improved

	Req ID1	Req ID2	Req ID3	Req ID4	Req ID5
Req ID1					
Req ID2					
Req ID3	X				
Req ID4			X		
Req ID5	X	X			

(a)

	DE ID1	DE ID2	DE ID3	DE ID 4
Req ID1	X			
Req ID2			X	
Req ID3		X		
Req ID4		X		
Req ID5				X

(b)

FIGURE 2. Simple example of traceability matrices: (a) shows which requirements depend upon which others—that is, Req ID3 depends upon Req ID1—and (b) shows which design elements satisfy which requirements—that is, Req ID3 is satisfied by DE ID2.

visualizations to render traces visible and support traceability-enabled tasks. However, a cursory look at any requirements management tool will show the continued dominance of the matrix, perhaps augmented by a graphic or hierarchical listing of salient dependencies between artifacts. We suggest that a major barrier to advancement with visualization in this area has been the uncertainty surrounding the nature of the hidden tracery structure that we've been striving to render visible in new ways. This limitation is compounded by an equally poor understanding of the intended end uses for such visual representations.

Tracery: A Pattern of Traces

When practitioners engineer requirements and their resulting software systems, they're creating—alongside and within all other important development artifacts—a web of interconnections that we wish to call requirements tracery. Revealing and understanding the

pattern of this tracery has proven critical in many other fields where tracing is fundamental (for example, ichnology and epidemiology), while some even explicitly design the pattern (for example, metrology). We've written about the practices of other tracing disciplines elsewhere.²

Some kind of semantic building blocks make up this requirements tracery but their exact and full arrangement is completely unknown. It's also unknown whether common patterns occur in such arrangements, something equivalent to the configurations of stonework in rose windows. Our best guess at the moment is that the pattern is composed of the multiple tracks that individual requirements leave as they become implemented. Some requirements split, some combine, some halt prematurely, and some suddenly appear or reappear for no discernable reason.

Our appeal through this column consists of four ideas. First, if we're

going to start paying attention to improving and exploiting the structure that supports our requirements and other development artifacts in software engineering, let's start by naming this structure. *Requirements tracery* is our proposal.

Second, let's begin to render requirements tracery visible and examine its pattern across different types of projects and contexts. As an artistic by-product of these efforts, the traceability community could even curate a gallery of our discipline's unique "rose windows."

Third, let's learn about the impact of various tracery patterns on the volatility and viability of managing requirements and on the support of other im-

portant traceability-enabled tasks.

Ultimately, let's use the insight gained to either inform the design of a requisite requirements tracery as an integral part of the software engineering process or help guide the recovery of the traces on demand later.

If our requirements tracery is created well, it will be an engineered structure, purposefully crafted and exposable as needed. Its presence in end products will be as imperceptible to end users as the stonework to viewers of rose windows—in the process, perhaps our software systems will acquire some of the windows' splendor and longevity. 

References

1. H. Chisholm, *The Encyclopaedia Britannica: A Dictionary of Arts, Sciences, Literature and General Information*, 11th ed., vol. 27, 1911, pp. 115–116.
2. O.C.Z. Gotel and S.J. Morris, "Out of the Labyrinth: Leveraging Other Disciplines for Requirements Traceability," to be published in *Proc. 19th IEEE Int'l Conf. Requirements Eng.* (RE 11), IEEE CS Press, 2011.

OLLY GOTEL is an independent researcher working in New York City. Contact her at olly@gotel.net.

STEPHEN MORRIS is an honorary senior visiting research fellow in the Department of Computing at City University London. Contact him at sjm@soi.city.ac.uk.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

IEEE Software

IEEE Software (ISSN 0740-7459) is published bimonthly by the IEEE Computer Society. IEEE headquarters: Three Park Ave., 17th Floor, New York, NY 10016-5997. IEEE Computer Society Publications Office: 10662 Los Vaqueros Cir., Los Alamitos, CA 90720-1314; +1 714 821 8380; fax +1 714 821 4010. IEEE Computer Society headquarters: 2001 L St., Ste. 700, Washington, DC 20036. Subscription rates: IEEE Computer Society members get the lowest rate of US\$54 per year, which includes printed issues plus online access to all issues published since 1984. Go to www.computer.org to subscribe to order and for more information on other subscription prices. Back issues: \$20 for members, \$173.33 for nonmembers (plus shipping and handling).

Postmaster: Send undelivered copies and address changes to *IEEE Software*, Membership Processing Dept., IEEE Service Center, 445 Hoes Lane, Piscataway, NJ 08854-4141. Periodicals Postage Paid at New York, NY, and at additional mailing offices. Canadian GST #125634188. Canada Post Publications Mail Agreement Number 40013885. Return undeliverable Canadian addresses to PO Box 122, Niagara Falls, ON L2E 6S8, Canada. Printed in the USA.

Reuse Rights and Reprint Permissions: Educational or personal use of this material is permitted without fee, provided such use: 1) is not made for profit; 2) includes this notice and a full citation to the original work on the first page of the copy; and 3) does not imply IEEE endorsement of any third-party products or services. Authors and their companies are permitted to post the accepted version of IEEE-copyrighted material on their own web servers without permission, provided that the IEEE copyright notice and a full citation to the original work appear on the first screen of the posted copy. An accepted manuscript is a version which has been revised by the author to incorporate review suggestions, but not the published version with copy-editing, proofreading and formatting added by IEEE. For more information, please go to: http://www.ieee.org/publications_standards/publications/rights/paperversionpolicy.html. Permission to reprint/republish this material for commercial, advertising, or promotional purposes or for creating new collective works for resale or redistribution must be obtained from IEEE by writing to the IEEE Intellectual Property Rights Office, 445 Hoes Lane, Piscataway, NJ 08854-4141 or pubs-permissions@ieee.org. Copyright © 2011 IEEE. All rights reserved.

Abstracting and Library Use: Abstracting is permitted with credit to the source. Libraries are permitted to photocopy for private use of patrons, provided the per-copy fee indicated in the code at the bottom of the first page is paid through the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923.